

UPN, PASIÓN POR
TRANSFORMAR VIDAS

CURSO: BASE DE DATOS AVANZADAS Y BIG DATA

UPN.EDU.PE

Saberes previos



1. ¿Para que se usa BD noSQL?

2. ¿Qué es BI?

Fuente:https://www.bbc.com/mundo/noticias/2015/11/151106_preguntas_ciencia_basicas_padres_lp

Logro de la sesión



Al término de la sesión, el estudiante aplica enfoques innovadores en la gestión de datos con bases NoSQL y operaciones CRUD. Además, analiza las fases de un proyecto de BI y diseña un datawarehouse para optimizar la integración y transformación de datos en entornos empresariales.

Agenda



1. Nuevos paradigmas en el manejo de datos.Bases de datos NoSQL.

2. Operaciones CRUD en NoSQL.

3. Business intelligence, Fases de un proyecto BI.

1.Nuevos paradigmas de BD



Las bases de datos NoSQL, también conocidas como bases de datos personalizadas, están diseñadas para modelos de datos específicos y almacenan los datos en esquemas flexibles que se escalan con facilidad para aplicaciones modernas. Muchas cargas de trabajo de bases de datos pueden beneficiarse de la rentabilidad y el rendimiento de las bases de datos NoSQL. (Amazon Web Services, s.f.)

2. Ventajas de las bases de datos NoSQL



- **Flexibilidad**

Las bases de datos NoSQL generalmente ofrecen esquemas flexibles que permiten un desarrollo más rápido y más iterativo. El modelo de datos flexible hace que las bases de datos NoSQL sean ideales para datos semiestructurados y no estructurados.

- **Escalabilidad**

Las bases de datos NoSQL generalmente están diseñadas para escalar horizontalmente usando clústeres distribuidos de hardware, en lugar de escalar añadiendo servidores caros y sólidos. Algunos proveedores de la nube manejan estas operaciones en segundo plano, como un servicio completamente administrado.

- **Alto rendimiento**

Las bases de datos NoSQL están optimizadas para modelos de datos y patrones de acceso específicos. Esto permite un mayor rendimiento que si intentara lograr una funcionalidad similar con bases de datos relacionales.

- **Altamente funcional**

Las bases de datos NoSQL proporcionan API altamente funcionales y tipos de datos que están diseñados específicamente para cada uno de sus respectivos modelos de datos.

Desventajas

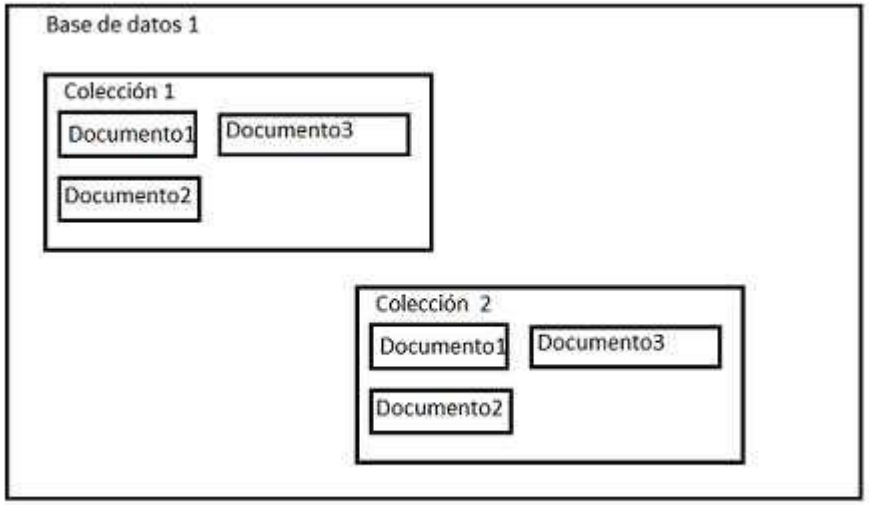


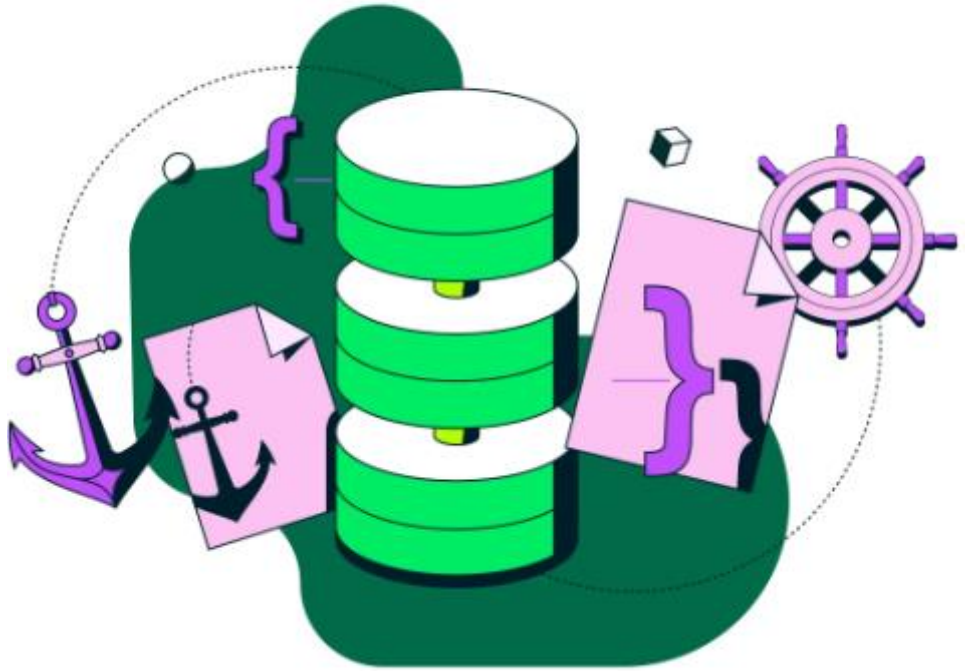
Desventajas:

- **Funciones de Fiabilidad:** esta si se quiere es una de las debilidades más grandes de los modelos **NoSQL**. Estas bases al no admitir funciones de fiabilidad lleva a los desarrolladores a implementar su propio código, lo que agrega mayor complejidad a los sistemas.
- **Aplicabilidad**: Esta desventaja está íntimamente relacionada con la anterior. La falta de inclusión de funciones de fiabilidad limita la aplicabilidad de estas BBDD a funciones delicadas como las del sector bancario, limitando mucho su potencial.
- **Incompatibilidad en consultas SQL:** en la mayoría de los casos las BBDD NoSQL son incompatibles a consultas SQL, lo que agrega una dificultad mayor ya que en caso de ser necesario se debe incluir una consulta manual, que puede hacer los procesos más lentos y complejos.

Mongo db

MongoDB es una base de datos documental, el elemento esencial es el documento que normalmente se los agrupa en colecciones de documentos similares. Una base de datos en MongoDB es un conjunto de colecciones:





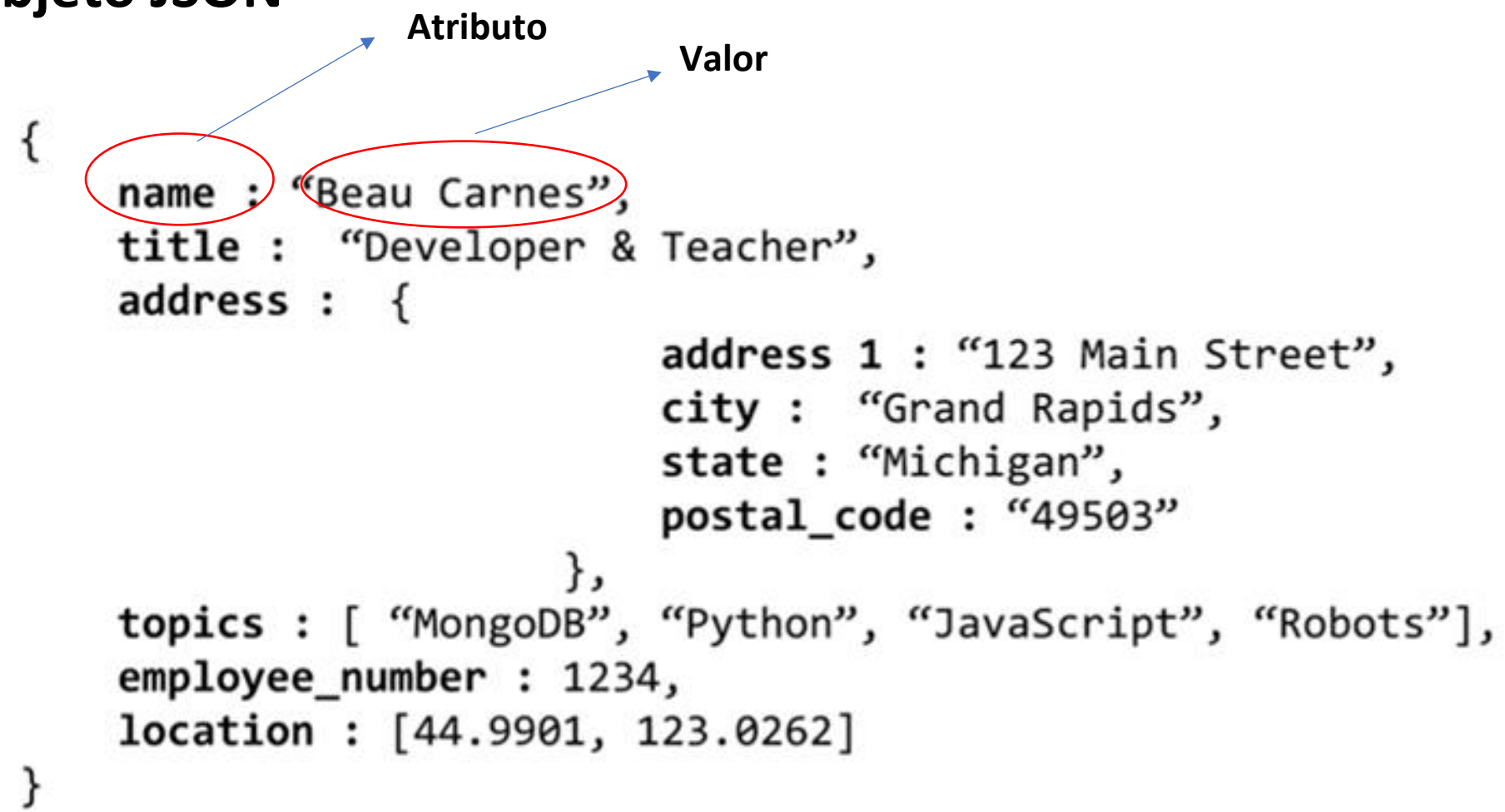
Comparación BD SQL vs MongoDB



Tabular (Relational)	MongoDB
Database	Database
Table	Collection
Row	Document
Index	Index
Join	$lookup
Foreign Key	Reference

2. BD no SQL

Objeto JSON



Fuente:https://www.jacobsoft.com.mx/en/como-crear-una-base-de-datos-con-mongodb-y-compass/

2. BD no SQL



```
{
    name : "Beau Carnes",
    title :  "Developer & Teacher",
    address : {
                    address 1 : "123 Main Street",
                    city :  "Grand Rapids",
                    state : "Michigan",
                    postal_code : "49503"
                },
    topics : [ "MongoDB", "Python", "JavaScript", "Robots"],
    employee_number : 1234,
    location : [44.9901, 123.0262]
}
```

String

Nested Document

Array

Integer

Geo-Spatial Coordinates

Fuente:https://www.jacobsoft.com.mx/en/como-crear-una-base-de-datos-con-mongodb-y-compass/

Seleccionar

```
db.coleccion.find({ filtro }, { proyeccion });
```

•**filtro**: (Opcional) Un objeto que define las condiciones para filtrar los documentos que quieres obtener.

•**proyeccion**: (Opcional) Un objeto que especifica qué campos deseas incluir o excluir en el resultado.

```
db.coleccion.find();
```

Esto devuelve todos los documentos en la colección.

```
db.usuarios.find({ edad: { $gte: 30 } });
```

Obtener todos los documentos donde el campo edad sea mayor o igual a 30:

Seleccionar



```
db.usuarios.find().limit(5);
```

Ordenar los usuarios por edad de manera ascendente:

```
db.usuarios.find().sort({ edad: 1 });
```

Ordenar los usuarios por edad de manera ascendente:

```
db.usuarios.findOne({ nombre: "Ana" });
```

Obtener el primer usuario cuyo nombre sea "Ana"

Inserción



Sintaxis:
db.coleccion.insertOne(documento);

```
db.usuarios.insertOne({
   nombre: "Ana",
   edad: 25
});
```

Inserta un solo documento en una colección.

Sintaxis:
db.coleccion.insertMany([documento1, documento2, ...]);

```
db.usuarios.insertMany([
   { nombre: "Carlos", edad: 28 },
   { nombre: "Lucía", edad: 32 },
   { nombre: "Pedro", edad: 24 }
]);
```

Inserta varios documentos en una colección al mismo tiempo

Actualización operadores



- **$set**: Se usa para cambiar el valor de un campo específico en el documento.
- **$inc**: Se usa para incrementar (o decrementar) el valor de un campo numérico.
- **$push**: Agrega un elemento a un arreglo.
- **$rename**: Cambia el nombre de un campo.

Actualización



Sintaxis:

```
db.coleccion.updateOne(
   { filtro },         // Condición para buscar el documento
   { actualización }    // Cambios que se quieren hacer
);
```

```
db.usuarios.updateOne(
   { nombre: "Ana" },  // Filtro: buscar al usuario con nombre "Ana"
   { $set: { edad: 26 } }  // Actualización: cambia su edad a 26
);
```

Actualización



```
db.coleccion.updateMany(
   { filtro },         // Condición para buscar los documentos
   { actualización }    // Cambios que se quieren hacer
);
```

```
db.usuarios.updateMany(
   { edad: { $gte: 30 } },  // Filtro: buscar usuarios con edad mayor o igual a 30
   { $inc: { edad: 1 } }    // Actualización: incrementar su edad en 1
);
```

Eliminación



```
db.coleccion.deleteOne({ filtro });
```

eliminará solo el primer documento que coincida con el filtro, aunque haya varios documentos que cumplan la condición.

```
db.usuarios.deleteOne({ nombre: "Ana" });
```

```
db.coleccion.deleteMany({ filtro });
```

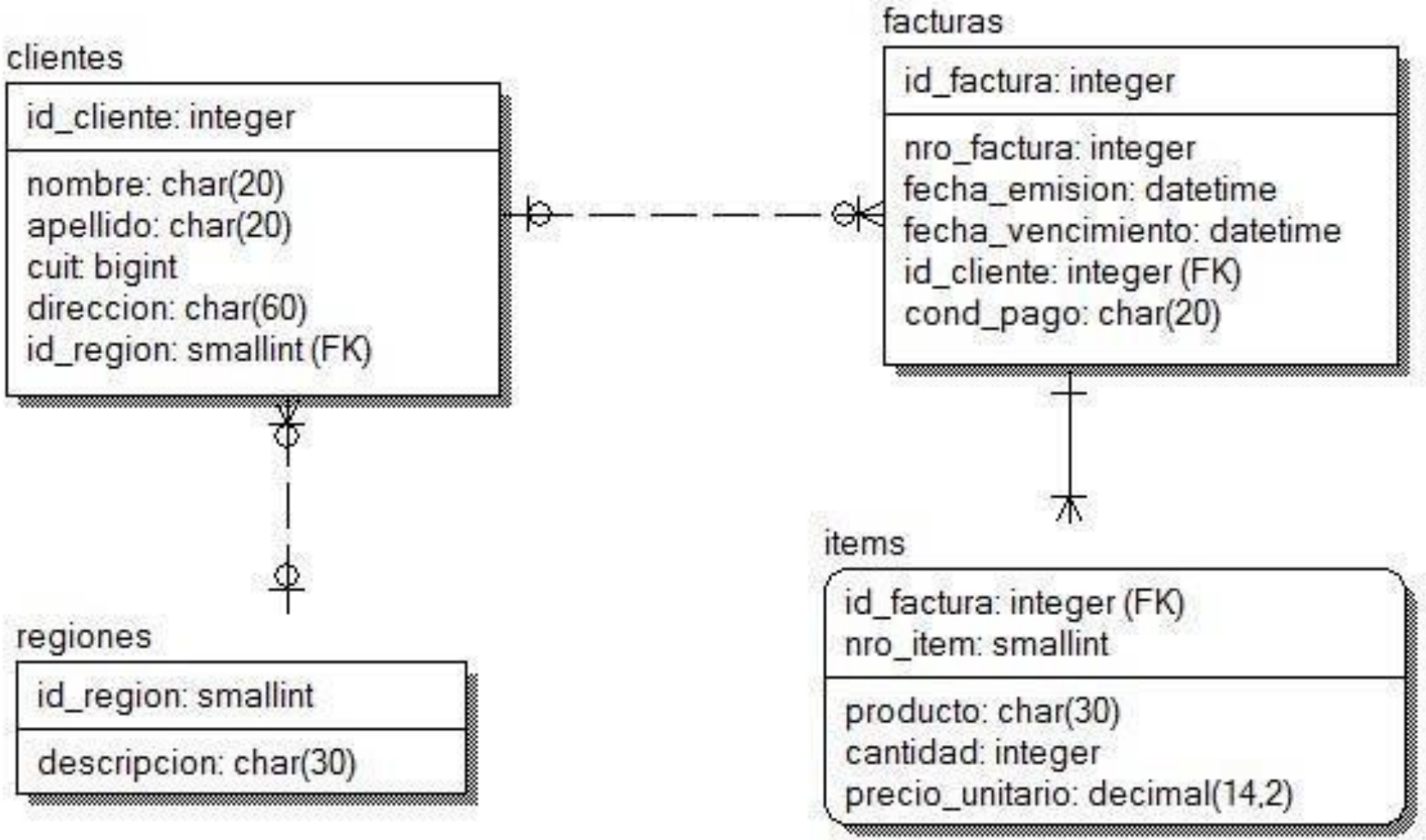
eliminará todos los documentos que cumplan con la condición dada.

```
db.usuarios.deleteMany({ edad: { $gte: 30 } });
```



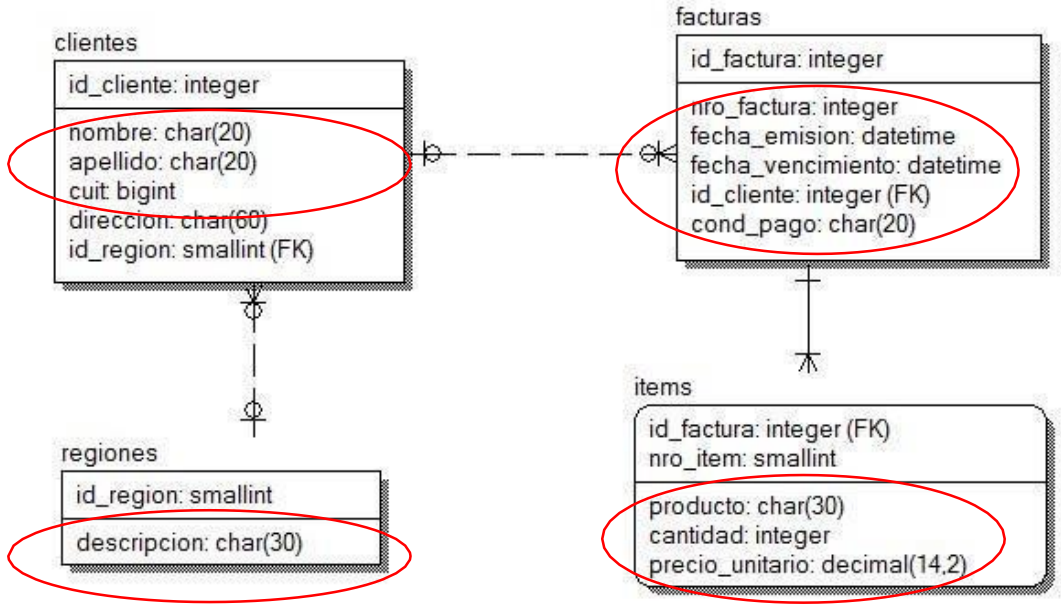
UTILIZANDO MONGODB

Caso Práctico



Caso Práctico

Armaremos un modelo que contenga la información de las facturas y todos sus ítems, detallando el nombre, apellido, cuit y región del cliente al que se le emitió la factura, para poder realizar consultas desde un portal de facturas de la forma más performante posible.



UTILIZACIÓN DE MONGODB DESDE LA LÍNEA DE COMANDOS

Operaciones sobre una colección

Ver todos los comandos:

db.facturas.help()

Ejemplos:

Ver todos los documentos:

db.facturas.find()

Cantidad de documentos en la colección:

db.facturas.count()

Espacio ocupado por los documentos de la colección:

db.facturas.dataSize()

Operaciones sobre una colección

Inicio de servidor mongo por consola:

C:>mongod

Inicio de cliente mongo por consola:

C:>mongo

Ver bases de datos

show dbs

Ver colecciones

show collections

Caso Práctico

Insertar

```
db.facturas.insert({"_id":
  ObjectId("55e4a6fabfc68c676a041063"),"cliente":{"apellido":"Malinez","cuit":2.740488484e+09,"nombre":"Marin
  a","region":"CENTRO"},"condPago":"CONTADO","fechaEmision": ISODate("2014-02-
  20T00:00:00.000Z"),"fechaVencimiento": ISODate("2014-02-
  20T00:00:00.000Z"),"item":[{"cantidad":11.0,"precio":18.0,"producto":" CORREA
  12mm"},{"cantidad":1.0,"precio":490.0,"producto":"TALADRO 12mm"}],"nroFactura":1000.0})
db.facturas.insert({"_id":
  ObjectId("55e4a6fbbfc68c676a041064"),"cliente":{"apellido":"Zavasi","cuit":2.038373771e+09,"nombre":"Martin",
  "region":"CABA"},"condPago":"30 Ds FF","fechaEmision": ISODate("2014-02-
  20T00:00:00.000Z"),"fechaVencimiento": ISODate("2014-03-
  22T00:00:00.000Z"),"item":[{"cantidad":2.0,"precio":134.0,"producto":"CORREA 10mm"}],"nroFactura":1001.0})
db.facturas.insert({"_id":
  ObjectId("55e4a6fbbfc68c676a041065"),"cliente":{"apellido":"Zavasi","cuit":2.038373771e+09,"nombre":"Martin",
  "region":"CABA"},"condPago":"CONTADO","fechaEmision": ISODate("2014-02-
  20T00:00:00.000Z"),"fechaVencimiento": ISODate("2014-02-
  20T00:00:00.000Z"),"item":[{"cantidad":6.0,"precio":60.0,"producto":"TUERCA
  2mm"},{"cantidad":12.0,"precio":134.0,"producto":"CORREA 10mm"}],"nroFactura":1002.0})
```

Operaciones sobre una colección

Buscar documento para cliente con determinado apellido

db.facturas.find({"cliente.apellido":"Malinez"})

Consultar los primeros dos documentos

db.facturas.find().limit(2)

Consultar documentos salteando los primeros dos documentos

db.facturas.find().skip(2)

Visualización mejorada

db.facturas.find().pretty()

Operaciones sobre una colección

Consultar dos documentos, salteando los dos primeros documentos de una colección, mostrándolos en un modo mejorado.

db.facturas.find().limit(2).skip(2).pretty()

Consultar documentos sólo mostrando algunos datos

db.facturas.find({"cliente.apellido":"Malinez"}, {"cliente.cuit":1, "cliente.region":1})

Buscar documento para cliente con dos criterios

Para Zavasi teníamos dos documentos: db.facturas.find({"cliente.apellido":"Zavasi"}).pretty()

Agregamos un criterio: db.facturas.find({"cliente.apellido":"Zavasi", "nroFactura":1001.0}).pretty()

Ordenamiento en forma ascendente

db.facturas.find().sort({nroFactura:1})

Ordenamiento en forma descendente

db.facturas.find().sort({nroFactura:-1})

Consultando una Colección – Criterios de Selección

Busca la cantidad de facturas cuyo Nro. de Factura sea mayor que 1465

Operadores $

```
db.facturas.find( { nroFactura : {$gt:1465} } ).count()
```

$gt
$gte
$lt
$lte
$not
$or
$in
$nin
$exist
$regex

```
>
> db.facturas.find( { nroFactura : {$gt:1465} } ).count()
30512
>
```

Busca las facturas cuya fecha de emisión sea mayor o igual al 24/02/2014.

```
db.facturas.find({fechaEmision:{$gte: ISODate("2014-02-24T00:00:00Z")} } )
```

```
> db.facturas.find({fechaEmision:{$gte: ISODate("2014-02-24T00:00:00Z")} } )
{ "_id" : ObjectId("53685dbb2baf7b93f61df567"), "nroFactura" : 1450, "fechaEmisi
on" : ISODate("2014-02-24T00:00:00Z"), "fechaVencimiento" : ISODate("2014-02-24T
00:00:00Z"), "condPago" : "CONTADO", "cliente" : { "nombre" : "Juan Manuel", "ap
ellido" : "Manoni", "cuit" : 2029889382, "region" : "NEA" }, "item" : [
{       "producto" : "TUERCA 2mm",      "cantidad" : 2,          "precio" : 60 },
        {       "producto" : "TALADRO 12mm",    "cantidad" : 1,          "precio"
 : 490 },       {       "producto" : "TUERCA 5mm",      "cantidad" : 15,
"precio" : 90 } ] }
{ "_id" : ObjectId("53685dbb2baf7b93f61df568"), "nroFactura" : 1451, "fechaEmisi
on" : ISODate("2014-02-24T00:00:00Z"), "fechaVencimiento" : ISODate("2014-03-26T
00:00:00Z"), "condPago" : "30 Ds FF", "cliente" : { "nombre" : "Soledad", "apell
ido" : "Lavagno", "cuit" : 2729887543, "region" : "NOA" }, "item" : [  {  "produ
```

Insertando un Documento

El método insert tiene la siguiente sintaxis:

Evalúa si existe un próximo documento. Devuelve True o False.

.

```
db.collection.insert
( <document or array of documents>,
  { writeConcern: <document>,
    ordered: <boolean> } )
```

writeconcern	Es opcional, lo veremos en la parte de consistencia.
Ordered	lo vemos en un par de slides

Ejemplo, inserción de un documento sin _id:

```
db.facturas.insert({nroFactura:30003,codPago:"CONTADO"})
```

_id: Document Id único autogenerado

```
> db.facturas.insert({nroFactura:30003,codPago:"CONTADO"})
WriteResult({ "nInserted" : 1 })
>
>
> db.facturas.find({nroFactura:30003})
{ "_id" : ObjectId("5459a129cc19250561ad5f82"), "nroFactura" : 30003, "codPago"
: "CONTADO" }
```

Insertando un Documento

Ejemplo, inserción de un documento con _id:

```
db.facturas.insert({_id:23094776, nroFactura:30004,codPago:”CONTADO”})
```

```
> db.facturas.insert({_id:23094776,nroFactura:30004,codPago:"CONTADO"})
WriteResult({ "nInserted" : 1 })
>
>
> db.facturas.find({nroFactura:30004})
{ "_id" : 23094776, "nroFactura" : 30004, "codPago" : "CONTADO" }
```

Al crear una colección, el motor de BD crea un índice único sobre el atributo _id.

```
> db.facturas.insert({_id:23094776,nroFactura:30004,codPago:"30dsFF"})
WriteResult({
        "nInserted" : 0,
        "writeError" : {
                "code" : 11000,
                "errmsg" : "insertDocument :: caused by :: 11000 E11000 duplicat
e key error index: finanzas.facturas.$_id_  dup key: { : 23094776.0 }"
        }
})
```

Borrando Documentos

Operación Remove

Sintaxis

```
db.<collection_name>.remove({criterio_de_eliminación})
```

Esta operación eliminará los documentos que cumplan con el criterio definido.

Warning: Remove es una operación de tipo multi-documento!!

Recomendación: Es conveniente antes de borrar hacer un find o un count para asegurarse lo que quiero borrar.

Ejemplo 1 – Borrado de TODOS LOS DOCUMENTOS de una colección

```
db.accesos.remove({})
```

Elimina **TODOS LOS ELEMENTOS** de una colección.

```
> db.accesos.remove({})
WriteResult({ "nRemoved" : 3 })
>
> db.accesos.find()
```

Borrando Documentos



Ejemplo 2 – Remove por clave primaria

```
db.updtst.remove({_id:100})
```

Elimina el documento cuyo _id sea 100 de la colección **updtst**.

```
> db.updtst.remove({_id:100})
WriteResult({ "nRemoved" : 1 })
>
> db.updtst.find()
{ "_id" : 300, "items" : [ 88, 99, 97 ] }
{ "_id" : 200 }
```

Ejemplo 3 – Remove por un criterio con múltiples documentos que aplican

```
db.updtst.remove({items:88})
```

```
> db.updtst.find()
{ "_id" : 300, "items" : [ 88, 99, 97 ] }
{ "_id" : 500, "items" : 88 }
{ "_id" : 200 }
{ "_id" : 400, "items" : [ 77, 88 ] }
> db.updtst.remove({items:88})
WriteResult({ "nRemoved" : 3 })
> db.updtst.find()
{ "_id" : 200 }
>
```

Modificando Documentos

Permite modificar uno o más documentos de una colección. Por default modifica sólo un documento.

```
db.coleccion.update ( {clausula_where},
                      {documento_o_expresión_a_modificar},
                      { upsert, multi, writeconcern}
                    )
```

upsert (true o false)Si está configurado en “True” significa que realizará un update si existe un documento que concuerda con el criterio, o un insert si no existe algún documento que concuerde con el criterio. El valor default es “false”, en este caso no realiza un insert cuando no existe documento que concuerde con el criterio.

multi (true o false) Es opcional. Si es configurado en true, el update realiza la actualización de multiples documentos que concuerdan con el criterio cláusula_where. Si es configurado en false, modifica solo un documento. El valor default es false. Sólo actúa en updates parciales con operadores $.

writeconcern Es opcional, lo veremos en la parte de consistencia.

Modificando Documentos

Update Totales/Completos

Se realiza el update del documento completo, reemplazando el mismo.

Update Parciales

Operadores

Operadores sobre cualquier atributo

$set	Permite modificar el valor de un atributo, o agregar un nuevo atributo al documento.
$unset	Permite eliminar un atributo de un documento.
$inc	Incrementa o decrementa el valor de un atributo (n ó –n)

Operadores sobre Arrays

$push	Agrega un elemento a un Array o crea un Array con un elemento.
$addtoSet	Agrega un elemento al Array solo si no existe en el Array.
$pushAll	Agrega varios elementos a un Array con los valores indicados o crea un Array con esos elementos. *(Operación Múltiple)*
$pop	Elimina un elemento de un Array por sus extremos, permitiendo eliminar el primer elemento (-1)o el último (1).
$pull	Elimina todos los elementos de un Array que contengan el valor indicado.
$pullAll	Elimina todos los elementos de un Array que contengan alguno de los valores indicados. *(Operación Múltiple)*

Modificando Documentos Completos

Update Totales/Completos

```
db.updtst.update({x:2},{"x" : 2, "y" : 999 })
```

Este comando reemplaza el primer documento encontrado por con valor x:2 por este otro en donde el elemento y:999, no tengo el control de cuál estoy modificando, lo correcto era modificar poniendo en el criterio el _id.

```
> db.updtst.update({x:2},{"x" : 2, "y" : 999 })
> db.updtst.find()
{ "_id" : ObjectId("536a8240793253ebed598065"), "x" : 1, "y" : 999 }
{ "_id" : ObjectId("536a8245793253ebed598066"), "x" : 2, "y" : 999 }
{ "_id" : ObjectId("536a8248793253ebed598067"), "x" : 2, "y" : 100 }
{ "_id" : ObjectId("536a824b793253ebed598068"), "x" : 2, "y" : 300 }
{ "_id" : ObjectId("536a8250793253ebed598069"), "x" : 3, "y" : 100 }
{ "_id" : ObjectId("536a8254793253ebed59806a"), "x" : 3, "y" : 200 }
{ "_id" : ObjectId("536a8257793253ebed59806b"), "x" : 3, "y" : 300 }
```

Modificando Documentos Parciales

Update Parciales

Ejemplo 1 – Operador $set – Modificación de un valor de un atributo existente

Dado el siguiente documento:

```
> db.updtst.insert({_id:100,x:10,y:100})
```

```
db.updtst.update({_id:100},{$set : {x:100}})
```

Realizará una modificación del valor de atributo x a 100

```
> db.updtst.find({_id:100})
{ "_id" : 100, "x" : 100, "y" : 100 }
```

Modificando Documentos Parciales

Update Parciales

Otro Ejemplo – Operador $set – Opción multi – Agregar un atributo en todos los documentos

```
db.updtst.update({x:2},{$set : {z:"NUEVO"}},{multi:true})
```

Este reemplaza en TODOS los documentos encontrados con valor x:2 agregando el atributo z:"NUEVO"

```
> db.updtst.update({x:2},{$set : {z:"NUEVO"}},{multi:true})
> db.updtst.find({x:2})
{ "_id" : ObjectId("536a8245793253ebed598066"), "x" : 2, "y" : 999, "z" : "NUEVO
" }
{ "_id" : ObjectId("536a8248793253ebed598067"), "x" : 2, "y" : 100, "z" : "NUEVO
" }
{ "_id" : ObjectId("536a824b793253ebed598068"), "x" : 2, "y" : 300, "z" : "NUEVO
" }
```

BD no sql



- ✓Apache Cassandra
- ✓CouchDB
- ✓Redis
- ✓Neo4j
- ✓Firebase
- ✓MongoDB



Fuente:https://www.gadsoftware.com/sistema-bi-business-intelligence

4. Business intelligence, Fases de un proyecto BI

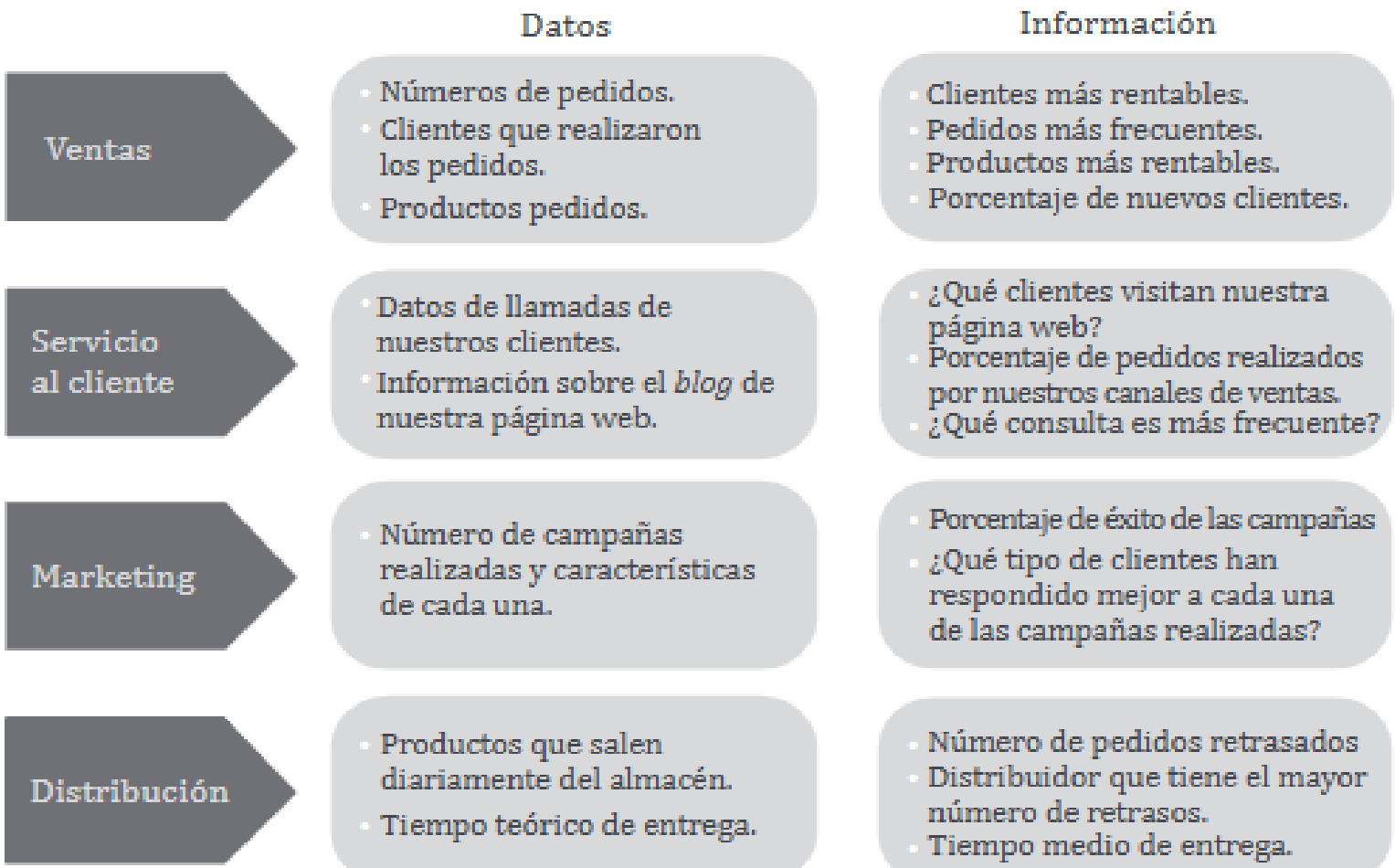
Business Intelligence se refiere al proceso de convertir datos en conocimiento y conocimiento en acciones para crear ventaja competitiva del negocio

The Data Warehouse Institute: instituto de educación en Business Intelligence y Data Warehouse. Citado en Libro Medina la Plata, E. (2012). Business Intelligence: Una guía práctica. Universidad Peruana de Ciencias Aplicadas (UPC)((pp. 25).

Business Intelligence o también llamado La Inteligencia de Negocios

Hace referencia al manejo optimizado de los datos que almacena, recopila y analiza una organización, siendo capaz de transformarlos en decisiones estratégicas que permitan el diseño de acciones orientadas a alcanzar el éxito empresarial.

Bussines Intelligence

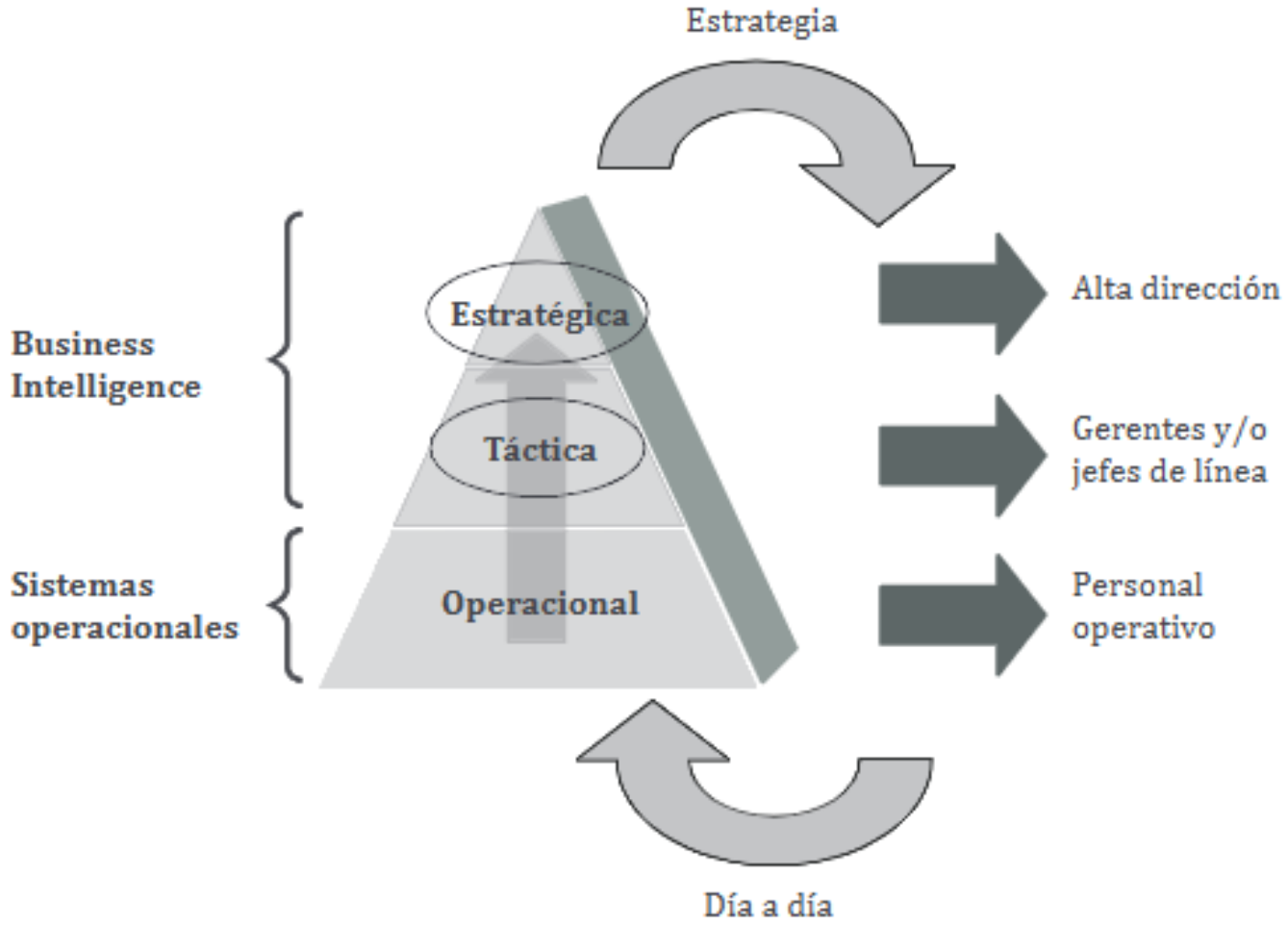


Fuente: Rodríguez, 2004.

Extraido: Medina la Plata, E. (2012). *Business Intelligence: Una guía práctica*. Universidad Peruana de Ciencias Aplicadas (UPC)(*(pp. 22)*. https://www.digitaliapublishing.com/a/45568

Bussines Intelligence

Gráfico 3.1. Necesidades de información en la empresa



Extraido: Medina la Plata, E. (2012). *Business Intelligence: Una guía práctica*. Universidad Peruana de Ciencias Aplicadas (UPC)(*(pp. 27)*. https://www.digitalapublishing.com/a/45568

Bussines Inteligence

***Business Intelligence* en el Nivel Estratégico**

- A este nivel, el *Business Intelligence* facilita y confiere a la alta dirección **contar con información, analizarla y monitorear las tendencias y evoluciones;** de esta manera puede valorar el nivel de consecución de los objetivos estratégicos de la organización, tomando rápidas decisiones.

***Business Intelligence* en el Nivel Táctico**

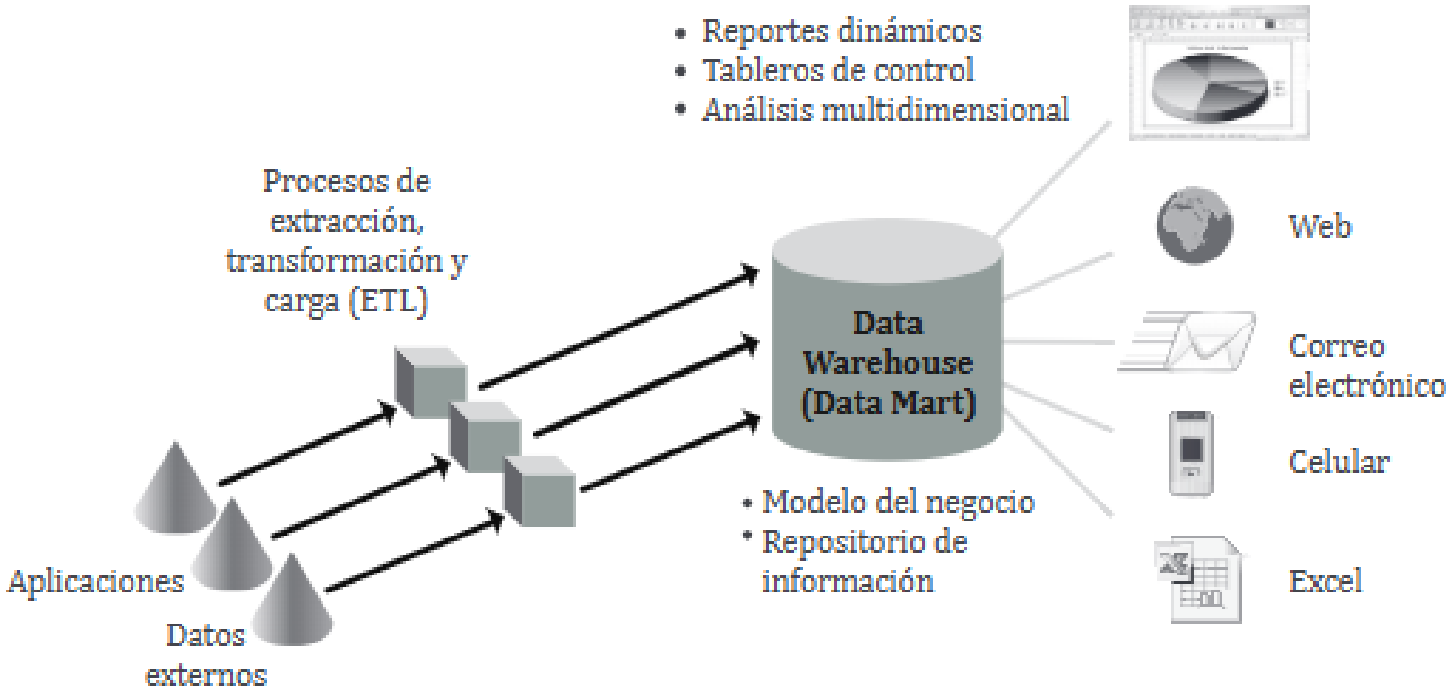
- El *Business Intelligence* permite que la gerencia que se encuentra en este nivel, junto con los analistas de datos, puedan hacer **uso de herramientas analíticas** para poder llegar a conclusiones, accediendo a información sin necesidad de recurrir a otros estamentos.

***Business Intelligence* en el Nivel Operativo**

- Facilita a los empleados información directa y clara sobre los procedimientos a seguir mediante instrucciones precisas. Esta información se representa mediante **informes con datos actualizados.**

Bussines Intelligence

Gráfico 3.2. Arquitectura de una solución de Business Intelligence



Elaboración propia.

Extraido: Medina la Plata, E. (2012). *Business Intelligence: Una guía práctica*. Universidad Peruana de Ciencias Aplicadas (UPC)(*(pp. 29)*. https://www.digitaliapublishing.com/a/45568

Ejemplos de aplicación

INDUSTRIA	PROYECTO DE BI	DESCRIPCIÓN
Retail	Análisis de Comportamiento del Cliente	Utilizar datos de ventas y comportamiento en línea para personalizar ofertas y mejorar ventas.
Finanzas	Gestión de Riesgos y Cumplimiento	Analizar transacciones para detectar fraudes y asegurar el cumplimiento de normativas.
Salud	Optimización de Rutas de Atención al Paciente	Mejorar la eficiencia de los servicios hospitalarios y la satisfacción del paciente.
Manufactura	Optimización de la Cadena de Suministro	Analizar y prever demandas para ajustar la producción y minimizar costos de inventario.
Educación	Análisis de Rendimiento Estudiantil	Seguimiento del progreso de los estudiantes y desarrollo de intervenciones educativas.
Telecomunicaciones	Análisis de Churn de Clientes	Identificar factores que contribuyen a la pérdida de clientes y desarrollar estrategias de retención.
Transporte	Optimización de Logística y Rutas	Mejorar las rutas de entrega y reducir costos operativos mediante el análisis de datos de GPS.
Energía	Predicción de Demanda y Gestión de Recursos	Prever la demanda energética para optimizar la generación y distribución de energía.
Gobierno	Análisis de Datos para Políticas Públicas	Utilizar datos demográficos y económicos para informar la creación de políticas.

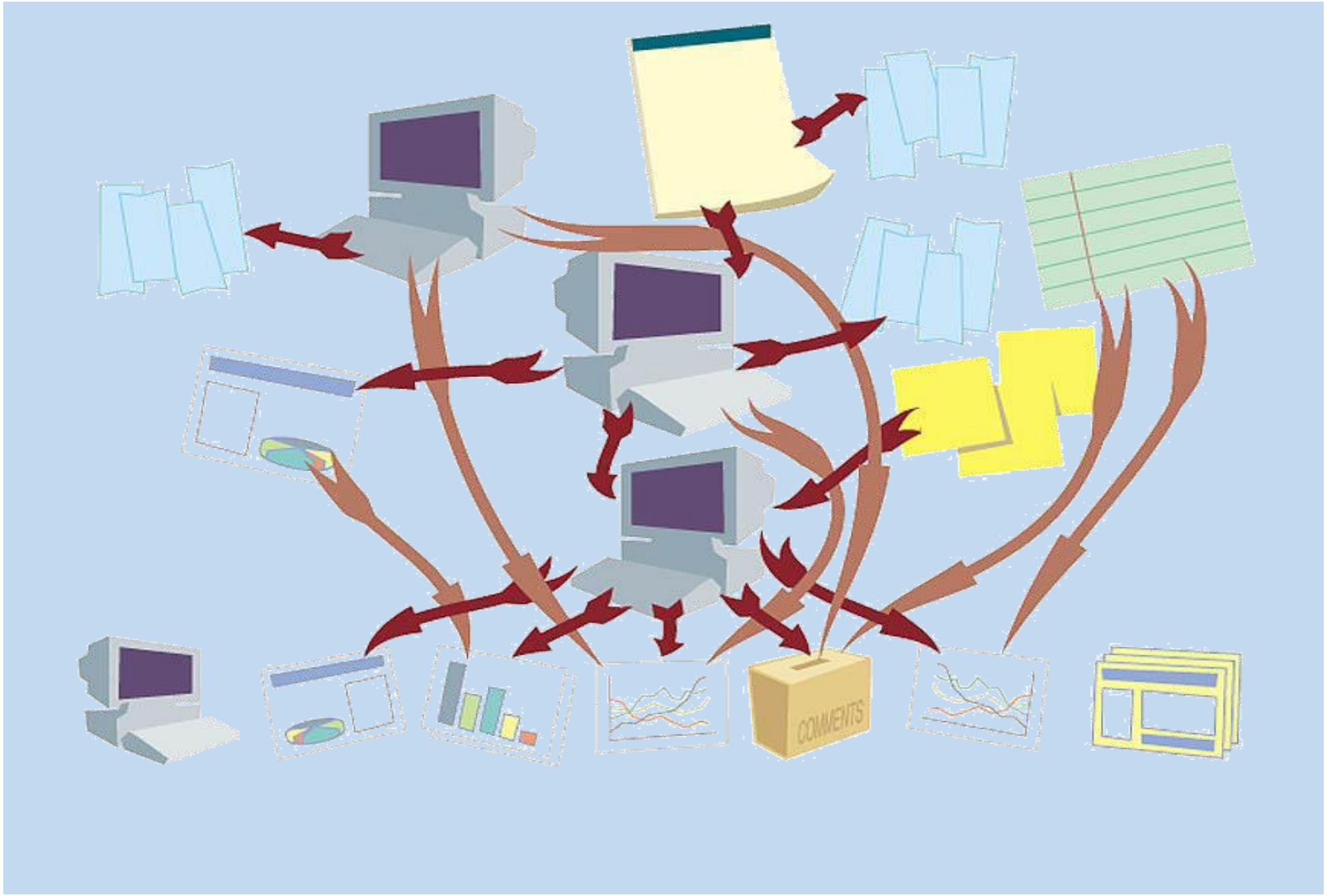
Fuente: Tecnología BI. (2025, February 26). *¿Cómo implementar Business Intelligence?* Tecnología BI. https://tecnologiabi.com/como-implementar-business-intelligence/

Tipo de implementación

Tipo de Implementación	Características
1. Tradicional (On-Premise)	- Infraestructura local. - Control total sobre sistemas y seguridad. - Alta inversión inicial.
2. En la Nube (Cloud-Based)	- Servicios alojados en la nube. - Escalabilidad y flexibilidad. - Actualizaciones automáticas.
3. Híbrida	- Combina On-Premise y Cloud. - Ofrece control y flexibilidad.
4. Modular o Faseada	- Implementación gradual por módulos. - Reducción de riesgos. - Adaptable a resultados.
5. Autoservicio (Self-Service)	- Empodera a los usuarios finales. - Rápida toma de decisiones. - Requiere cultura de datos.
6. Departamental	- Implementación en departamentos específ... - Soluciones personalizadas. - Expansión gradual.
7. Big Data	- Manejo de grandes volúmenes de datos. - Uso de tecnologías avanzadas (Hadoop, Spark, NoSQL).
8. Ágil	- Metodología iterativa. - Despliegue rápido. - Fomenta la colaboración entre usuarios y TI.
9. Personalizada	- Soluciones a medida. - Integración profunda con sistemas existentes. - Consultoría especializada.

Fuente: Tecnología BI. (2025, February 26). *¿Cómo implementar Business Intelligence?* Tecnología BI. https://tecnologiabi.com/como-implementar-business-intelligence/

Parálisis de análisis. El problema: Información no integrada



Fases de un proyecto BI



1. **Identificar objetivos de negocio**

Establecen los objetivos y se realiza el análisis de las necesidades que requiere la organización para alcanzar dichos objetivos

2.- Identificar los orígenes de los datos

Donde están los datos?, revisar los datos de origen, comprender la estructura, el contenido y las interrelaciones, e identificar el potencial de los proyectos de datos

3.- Diseño del modelo de datos analítico

Confeccionar los modelos que nos van a permitir crear después los análisis de los objetivos de negocio. Estos modelos de datos pueden ser en **estrella o en copo de nieve**

4. Implementación del modelo de datos

En esta parte se lleva cabo un buen modelo de datos, algo muy importante para alinear el conocimiento de los analistas de datos con el lenguaje de la empresa.

6.Publicación y formación de los usuarios

Para que se obtenga el máximo rendimiento del proyecto de BI, los usuarios tienen que recibir el soporte y la formación necesaria para que puedan utilizar las herramientas e interpretar la información valiosa obtenida.

7. Profundizar en el análisis. La cultura del dato

por el porqué de los resultados y que encontremos las cuestiones que tenemos que enderezar. Y también que generemos un nuevo análisis par hacer un seguimiento del problema

Reconociendo la necesidad de BI



La organización no puede determinar la rentabilidad precisa de un cliente o de un producto

- Se cuestiona la rentabilidad.
- Las tendencias en ventas o de los hábitos de compra del cliente no pueden determinarse.

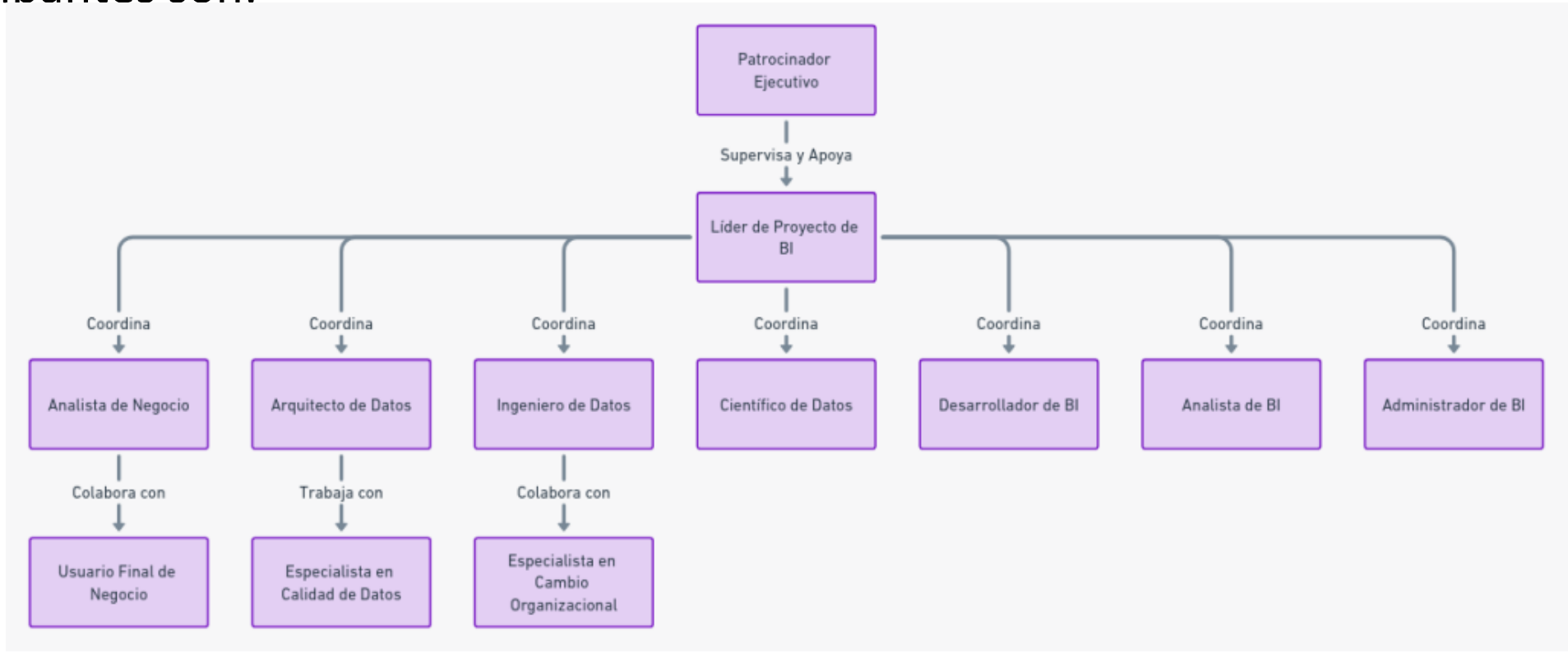
La organización no puede llegar a los clientes adecuadamente

- La segmentación es imposible.
- La perfilación de los clientes es imposible.

La organización está perdiendo participación en el mercado frente a los competidores

Equipo para el desarrollo de un proyecto BI

La implementación de un proyecto de BI requiere un equipo multidisciplinario, donde cada rol es fundamental para el éxito del proyecto. Los principales participantes son:



Fuente: https://tecnologiabi.com/como-implementar-business-intelligence/#Quienes_participan_en_la_implementacion_de_un_proyecto_de_Business_Intelligence

Problemas de la implementación de BI



- ✓ Resistencia al cambio
- ✓ Calidad de datos deficiente
- ✓ Integración de datos compleja
- ✓ Falta de compromiso de la alta dirección
- ✓ Expectativas irreales
- ✓ Falta de habilidades técnicas
- ✓ Problemas de escalabilidad
- ✓ Seguridad de datos y cumplimiento
- ✓ Deficiencias en la gestión de proyectos
- ✓ Falta de cultura de datos
- ✓ Selección de herramientas inadecuadas
- ✓ Infraestructura tecnológica inadecuada
- ✓ Dificultades en la adopción de usuarios
- ✓ Falta de mantenimiento y soporte

BI vs Análisis de datos



- **BI** se centra en recopilar, procesar y presentar datos de manera accesible para facilitar la toma de decisiones estratégicas. Está orientado a ejecutivos y tomadores de decisiones.

Fuente:https://www.gadsoftware.com/sistema-bi-business-intelligence

- **Análisis de datos** profundiza en la exploración y comprensión de los datos, identificando patrones, tendencias y anomalías. Es utilizado principalmente por analistas y científicos de datos.

En resumen, BI es más sobre la visualización y accesibilidad de la información, mientras que el análisis de datos se enfoca en la interpretación y el descubrimiento de conocimientos dentro de los datos.

BI vs IA

Principales aportes de la IA en BI:

- ✓ Análisis predictivo y prescriptivo
- ✓ Automatización de tareas
- ✓ Procesamiento del lenguaje natural (NLP)
- ✓ Descubrimiento de patrones
- ✓ Segmentación y personalización
- ✓ Recomendaciones y alertas
- ✓ Optimización de consultas

Conclusiones

¿Qué aprendimos hoy?

Referencia bibliográfica

Jacobsoft. (s.f.). *Cómo crear una base de datos con MongoDB y Compass*. Jacobsoft. Recuperado el 25 de febrero de 2025, de https://www.jacobsoft.com.mx/en/como-crear-una-base-de-datos-con-mongodb-y-compass/

Tutoriales de Programación YA. (s.f.). *Introducción a MongoDB: Concepto de base de datos en MongoDB*. Tutoriales de Programación YA. Recuperado el 25 de febrero de 2025, de https://www.tutorialesprogramacionya.com/mongodbya/detalleconcepto.php?punto=3&codigo=3&inicio=0

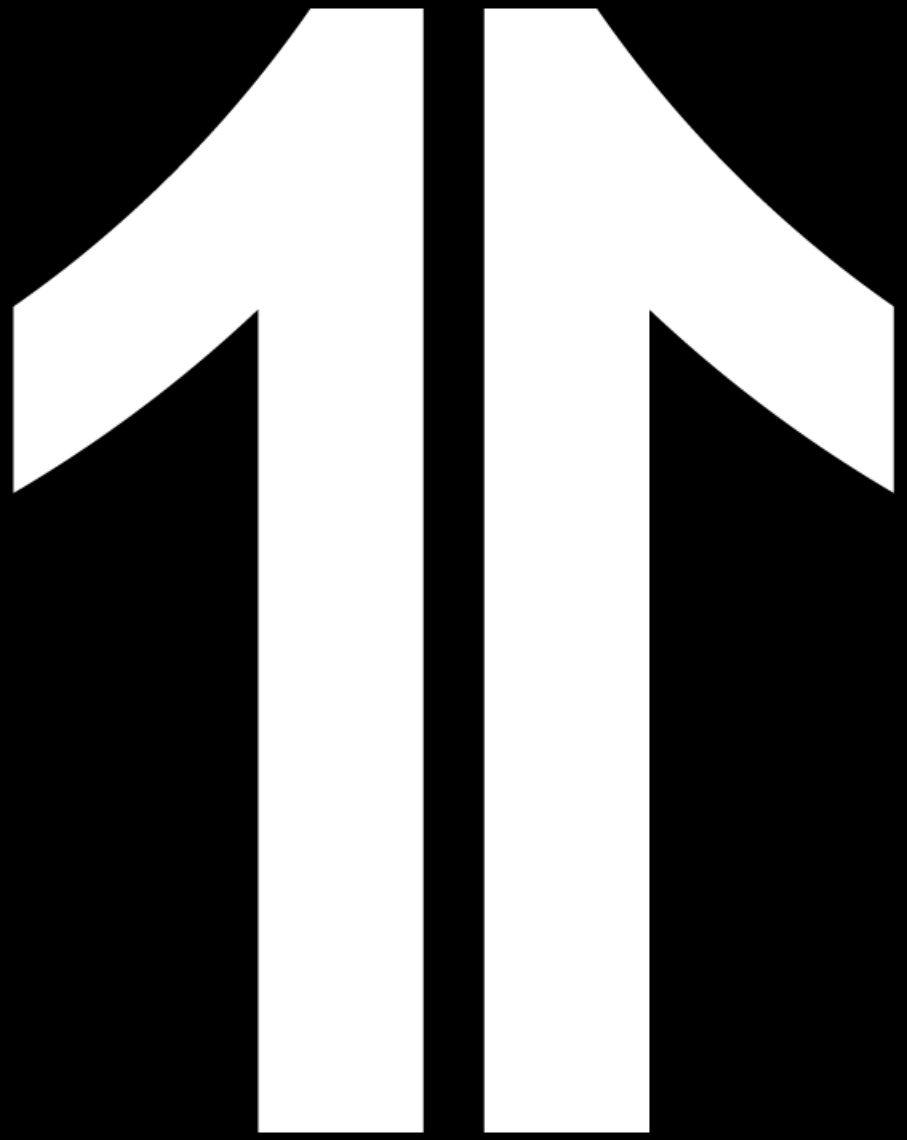
MongoDB, Inc. (s.f.). *Delete documents in MongoDB using Go drivers*. MongoDB. Recuperado el 25 de febrero de 2025, de https://www.mongodb.com/docs/drivers/go/current/fundamentals/crud/write-operations/delete/

López Benítez, Y. (2018). *Business intelligence : ADGG102PO* (1st ed.). IC Editorial.

Referencia bibliográfica

Tecnología BI. (2025, February 26). *¿Cómo implementar Business Intelligence?* Tecnología BI. https://tecnologiabi.com/como-implementar-business-intelligence/

GAD Software. (s.f.). *Sistema BI: Business Intelligence*. GAD Software. https://www.gadsoftware.com/sistema-bi-business-intelligence



GRACIAS


UPN
UNIVERSIDAD
PRIVADA
DEL NORTE